

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS: 3

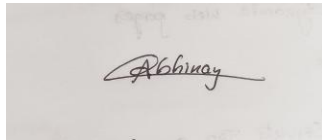
Total Marks: 30

Code of Conduct:

I ____ G. Abhinay (192373062) ____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1

A hospital management system requires a SOAP-based web service to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a SOAP-based web service for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

Answer to Question 1:

A) Structure of SOAP Messages:

A Simple Object Access Protocol (SOAP) message is an XML document containing the following elements:

1. SOAP Envelope: The root element of the XML document that defines the XML namespace and identifies the document as a SOAP message.
2. SOAP Header (Optional): Contains metadata such as security credentials, routing instructions, and session details.

3. SOAP Body: Contains the actual payload representing the RPC call request or response parameters.
4. SOAP Fault (Optional, inside Body): Provides error status and detail information if request processing fails.

B) SOAP Request Message Example (addPatient):

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:hos="http://www.simats.edu/hospital">
  <soap:Header>
    <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-
200401-wss-wssecurity-secext-1.0.xsd">
      <wsse:UsernameToken>
        <wsse:Username>adminDept</wsse:Username>
        <wsse:Password>departmentSecretPass</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </soap:Header>
  <soap:Body>
    <hos:addPatientRequest>
      <hos:patientName>John Doe</hos:patientName>
      <hos:age>35</hos:age>
      <hos:department>Cardiology</hos:department>
    </hos:addPatientRequest>
  </soap:Body>
</soap:Envelope>
```

C) Data Exchange Illustration:

1. Client (e.g., Reception App) wraps patient details in a SOAP XML Request envelope and sends it via HTTP POST.
2. Server (Hospital Server) receives the payload, authenticates via Header tokens, and parses the XML in the Body.
3. Server processes registration database changes and returns a SOAP XML Response (e.g., containing patient ID).

D) Justification of SOAP in Hospital Scenario:

- Security: SOAP supports WS-Security, allowing message-level encryption and signing. This guarantees patient record privacy even if the transport layer is compromised.
- Reliability: Built-in support for WS-ReliableMessaging ensures end-to-end delivery guarantees (exactly-once semantics), which is critical for medical safety.
- Interoperability: Enforcing strict XML Schema validations prevents data translation mismatches across heterogeneous legacy systems.

Question 2

An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use WSDL (Web Services Description Language) to describe the service.

Design the WSDL structure for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

Answer to Question 2:

A) WSDL Key Components:

1. **<types>**: Defines data types used in messages using XML Schema (XSD).
2. **<message>**: Defines the abstract data parameters transmitted (requests and responses).
3. **<portType>**: Defines abstract operations (methods) and messages mapping to them.
4. **<binding>**: Binds portType abstract interfaces to concrete physical protocols (typically SOAP over HTTP).
5. **<service>**: Specifies the physical web address endpoint (URL) where the service is accessed.

B) WSDL Structure Design (HospitalService.wsdl):

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="HospitalService"
    targetNamespace="http://www.simats.edu/hospital"
    xmlns="http://schemas.xmlsoap.org/wsdl/"
    xmlns:tns="http://www.simats.edu/hospital"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema"
    xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/">

    <types>
        <xsd:schema targetNamespace="http://www.simats.edu/hospital">
            <xsd:element name="GetPatientRequest" type="xsd:string"/>
            <xsd:element name="GetPatientResponse" type="xsd:string"/>
        </xsd:schema>
    </types>

    <message name="GetPatientReqMsg">
        <part name="parameters" element="tns:GetPatientRequest"/>
    </message>
    <message name="GetPatientRespMsg">
        <part name="parameters" element="tns:GetPatientResponse"/>
    </message>

    <portType name="HospitalPortType">
```

```

        <operation name="getPatientRecord">
            <input message="tns:GetPatientReqMsg"/>
            <output message="tns:GetPatientRespMsg"/>
        </operation>
    </portType>

    <binding name="HospitalSOAPBinding" type="tns:HospitalPortType">
        <soap:binding style="document"
transport="http://schemas.xmlsoap.org/soap/http"/>
        <operation name="getPatientRecord">
            <soap:operation
soapAction="http://www.simats.edu/hospital/getPatientRecord"/>
            <input><soap:body use="literal"/></input>
            <output><soap:body use="literal"/></output>
        </operation>
    </binding>

    <service name="HospitalService">
        <port name="HospitalPort" binding="tns:HospitalSOAPBinding">
            <soap:address location="http://localhost:8080/services/hospital"/>
        </port>
    </service>
</definitions>

```

C) Importance in Service Integration:

WSDL acts as a contract between provider and consumer. Client systems read the WSDL file to auto-generate proxy classes (using tools like wsimport), guaranteeing data structure alignment, target URL visibility, and message protocols, which streamlines enterprise system integration.

Question 3

A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

Answer to Question 3:

A) Analysis of Potential SOAP Issues:

1. Message Formatting Errors: Malformed XML syntax (e.g. unclosed tags, incorrect XML declaration), causing server XML parsers to abort.

2. Incorrect Namespaces: The server rejects requests if namespaces in the Envelope/Body tags do not exactly match the schemas defined in WSDL.

3. Endpoint Connectivity: SOAP calls hitting expired server ports, incorrect path routing, or blocked firewall ports, causing network timeouts or HTTP 404/503 errors.

B) Proposed Debugging Approach:

1. Network Traffic Monitoring: Use network sniffing tools (Fiddler, Wireshark, SoapUI) to capture active HTTP request and response XML payloads.

2. XML Schema (XSD) Validation: Validate message structures against target XSD schemas to identify structural syntax errors.

3. Logging SOAP Fault Envelopes: Ensure backend application catch blocks return standard SOAP Fault payloads to help trace error stacktraces:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Client</faultcode>
      <faultstring>Schema validation error: Missing required tag
'patientName'</faultstring>
      <detail>
        <errorCode>ERR-40012</errorCode>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

C) How the Corrected System Ensures Reliability:

- Schema Enforcement: Active schema check policies reject malformed messages early, saving backend processing bandwidth.

- Actionable Errors: Clean SOAP Fault envelopes tell clients why their request failed, enabling quick corrective retries.

- Endpoint Health Checks: Circuit breakers verify endpoint availability to avoid calling offline services, guaranteeing stable communication.

**RUBRICS FOR CALCULATION:
Industry Problem Task**

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation